AUTOMATION

Keep Claude working toward a goal

Set a completion condition with /goal and Claude keeps working across turns until the condition is met.

`/goal` requires Claude Code v2.1.139 or later.

The `/goal` command sets a completion condition and Claude keeps working toward it without you prompting each step. After each turn, a small fast model checks whether the condition holds. If not, Claude starts another turn instead of returning control to you. The goal clears automatically once the condition is met.

Use a goal for substantial work with a verifiable end state:

- Migrating a module to a new API until every call site compiles and tests pass
- Implementing a design doc until all acceptance criteria hold
- Splitting a large file into focused modules until each is under a size budget
- Working through a labeled issue backlog until the queue is empty

Compare ways to keep a session running

Three approaches keep the current session running between prompts. Pick based on what should start the next turn:

Approach	Next turn starts when	Stops when
`/goal`	The previous turn finishes	A model confirms the condition is met
`/loop`	A time interval elapses	You stop it, or Claude decides the work is done
Stop hook	The previous turn finishes	Your own script or prompt decides

`/goal` and a Stop hook both fire after every turn. `/goal` is a session-scoped shortcut: you type a

condition and it's active for the current session only. A Stop hook lives in your settings file, applies to every session in its scope, and can run a script for deterministic checks or a prompt for model-evaluated ones.

Auto mode on its own approves tool calls within a single turn but doesn't start a new one. Claude stops when it judges the work done. `/goal` adds a separate evaluator that checks your condition after every turn, so completion is decided by a fresh model rather than the one doing the work. The two are complementary: auto mode removes per-tool prompts, and `/goal` removes per-turn prompts.

> The approaches above keep the current session running. You can also schedule work that runs independent of any open session, such as nightly tests or morning triage. See **scheduling options** for cloud routines and desktop scheduled tasks.

Use `/goal`

One goal can be active per session. The same command sets, checks, and clears it depending on the argument.

Set a goal

Run `/goal` followed by the condition you want satisfied. If a goal is already active, the new one replaces it.

```
/goal all tests in test/auth pass and the lint step is clean
```

Setting a goal starts a turn immediately, with the condition itself as the directive. You don't need to send a separate prompt. While the goal is active, a `◎ /goal active` indicator shows how long the goal has been running.

After each turn, the evaluator returns a short reason explaining why the condition is or isn't met. The most recent reason appears in the status view and in the transcript so you can see what Claude is working toward next.

> A goal keeps running until the condition is met or you run `/goal clear`. Run `/goal` with no argument to see turns and tokens spent so far.

Write an effective condition

The **evaluator** judges your condition against what Claude has surfaced in the conversation. It doesn't run commands or read files independently, so write the condition as something Claude's own output can demonstrate. "All tests in `test/auth` pass" works because Claude runs the tests and the result lands in the transcript for the evaluator to read.

A condition that holds up across many turns usually has:

- **One measurable end state**: a test result, a build exit code, a file count, an empty queue
- **A stated check**: how Claude should prove it, such as "`npm test` exits 0" or "`git status` is clean"
- **Constraints that matter**: anything that must not change on the way there, such as "no other test file is modified"

The condition can be up to 4,000 characters.

To bound how long a goal runs, include a turn or time clause in the condition, such as `or stop after 20 turns`. Claude reports progress against that clause each turn and the evaluator judges it from the conversation.

Check status

Run `/goal` with no arguments to see the current state.

```
/goal
```

If a goal is active, the status shows:

- The condition
- How long it has been running
- How many turns have been evaluated
- The current token spend
- The evaluator's most recent reason

If no goal is active but one was achieved earlier in the session, the status shows the achieved condition along with its duration, turn count, and token spend.

Clear a goal

Run `/goal clear` to remove an active goal before its condition is met.

```
/goal clear
```

`stop`, `off`, `reset`, `none`, and `cancel` are accepted as aliases for `clear`. Running `/clear` to start a new conversation also removes any active goal.

Resume with an active goal

A goal that was still active when a session ended is restored when you resume that session with `--resume` or `--continue`. The condition carries over, but the turn count, timer, and token-spend baseline all reset on resume. A goal that was already achieved or cleared is not restored.

Run non-interactively

`/goal` works in **non-interactive mode**, in the **desktop app**, and through **Remote Control**. Setting a goal with `-p` runs the loop to completion in a single invocation:

```
claude -p "/goal CHANGELOG.md has an entry for every PR merged
this week"
```

Interrupt the process with Ctrl+C to stop a non-interactive goal before the condition is met.

How evaluation works

`/goal` is a wrapper around a session-scoped **prompt-based Stop hook**. Each time Claude finishes a turn, the condition and the conversation so far are sent to your configured **small fast model**, which defaults to Haiku. The model returns a yes-or-no decision and a short reason. A “no” tells Claude to keep working and includes the reason as guidance for the next turn. A “yes” clears the goal and records an achieved entry in the transcript.

The evaluator runs on whichever provider your session is configured for. It does not call tools, so it can only judge what Claude has already surfaced in the conversation.

Evaluation tokens are billed on the small fast model configured for your provider and are typically negligible compared to main-turn spend.

Requirements

`/goal` runs only in workspaces where you have accepted the trust dialog, because the evaluator is part of the hooks system. `/goal` is also unavailable when `disableAllHooks` is set at any settings level or when `allowManagedHooksOnly` is set in managed settings. In each case, the command tells you why instead of silently doing nothing.

See also

- **Run a prompt repeatedly with `/loop`**: re-run on a time interval instead of until a condition holds
- **Prompt-based hooks**: write your own Stop hook when you need custom evaluation logic
- **Auto mode**: approve tool calls automatically so each goal turn runs unattended
- **Scheduling comparison**: run work on a schedule independent of any open session